

L'épopée des silences

Spécification de transition V1 vers V2 - backend legacy, services cibles et stratégie d'extraction

Document complémentaire aux SFD v2

Ce document précise la réflexion de migration entre le backend RPG_ESI07 v1 et l'architecture cible de L'épopée des silences v2. Il complète les SFD principales en détaillant les découpages, les risques, les procédures, les comparaisons et les décisions de migration.

Métadonnée	Valeur
Projet	L'épopée des silences
Ancien nom	RPG_ESI07
Objet	Transition technique et fonctionnelle V1 -> V2
Périmètre	Backend legacy, Game Engine, Catalog, Player, Identity, Audit/GDPR, Leaderboard
Statut	Document de cadrage migration / pré-Catalog
Commit associé	docs(v2): map legacy backend domains to target services

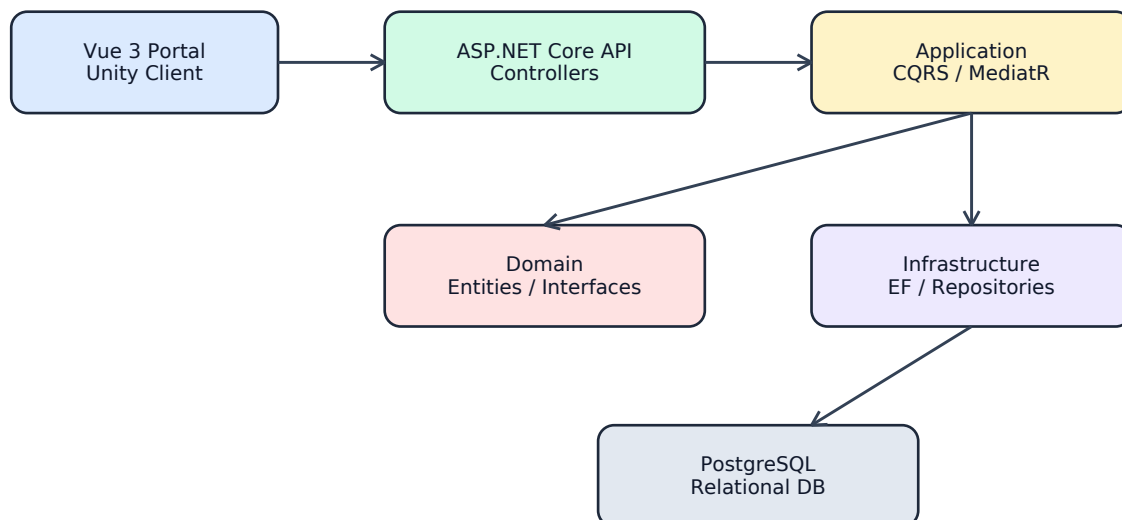
1. Synthèse décisionnelle

Le backend v1 n'est pas un brouillon à supprimer. Il constitue une base métier riche : entités, endpoints, handlers, repositories, services de sécurité, logique de scaling, tests, RGPD et audit. La stratégie correcte est une extraction progressive par bounded context.

La V2 ne doit pas créer un microservice par table. Elle doit conserver un Game Engine central pour le runtime gameplay, puis isoler les domaines périphériques : Catalog, Player, Identity, Audit/GDPR et Leaderboard.

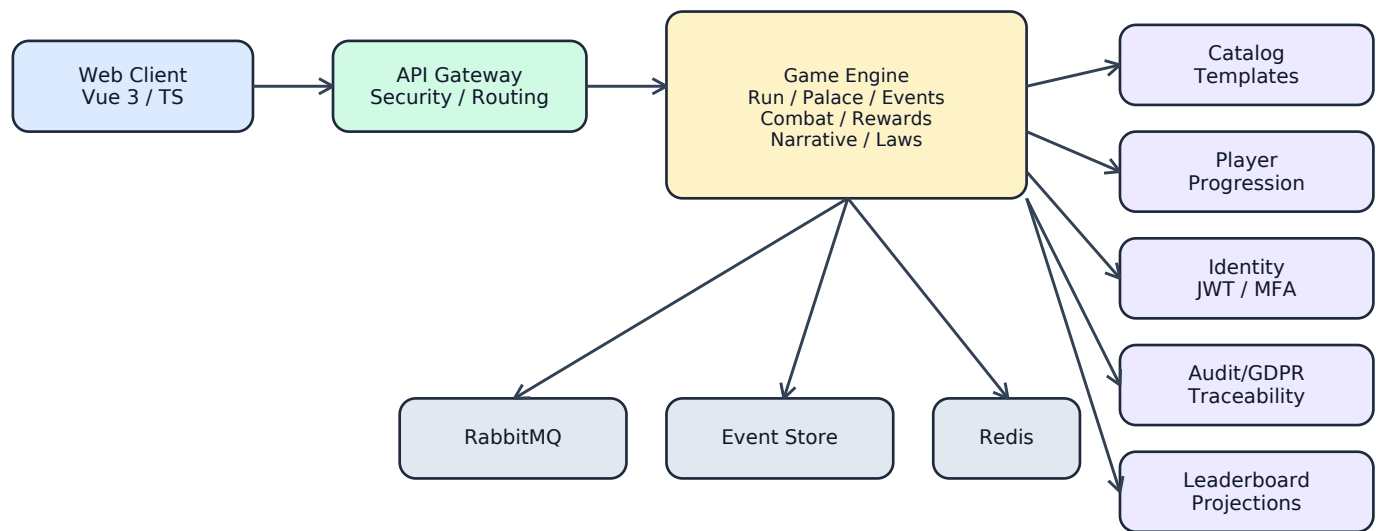
Décision	Justification	Conséquence
Conserver le backend v1 comme référence	Il contient déjà les modèles et cas d'usage métier	On mappe avant de recoder
Créer Catalog avant Combat MVP	Le combat dépend des ennemis, compétences et items	Évite de recoller du CRUD legacy dans Game Engine
Ne pas migrer GameSave tel quel	La v2 cible Event Sourcing sur les runs	Remplacer par Event Store + projections
Découper CombatStats	En v1 ce sont des statistiques cumulées	Créer PlayerCombatStatistics et CombatInstance séparément
Game Engine fort, pas microservices partout	Run, events, combat, rewards et lois sont fortement couplés	Cohérence runtime et serveur-autoritaire

Architecture V1 observée



2. Architecture cible V2

L'architecture cible place le Game Engine au centre du runtime de jeu. Les référentiels de contenu, l'identité, la progression durable, la conformité et les projections sont isolés dans des services spécialisés.



Service cible	Responsabilité principale	Données ou modèles clés
Game Engine	Runtime serveur-autoritaire	Run, Room, Node, CombatInstance, RewardOffer, ActivePalaceLaws
Catalog	Contenu versionné	EnemyTemplate, SkillTemplate, ItemTemplate, NpcTemplate, PalaceLawDefinition
Player	Progression durable	PlayerProfile, Inventory, Skills, Bestiary, Companions, PlayerCombatStatistics
Identity	Comptes et sécurité	User, JWT, MFA, Argon2id, roles, logout
Audit/GDPR	Traçabilité et conformité	AuditLog, consentements, export, anonymisation
Leaderboard	Classement et saisons	Run score, seed, season, projections

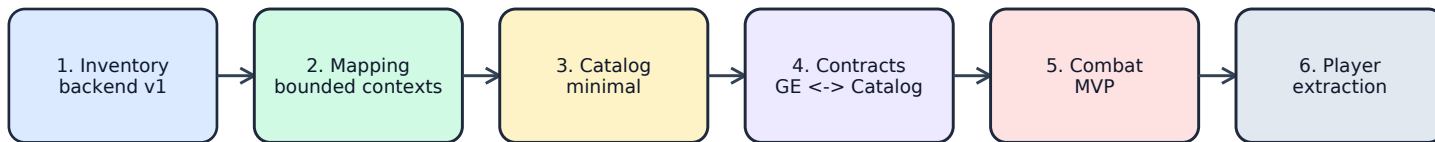
3. Mapping macro V1 vers V2

Élément legacy	Service cible	Décision	Commentaire
User / Auth / MFA	Identity	Migrer	Bonne base sécurité, à extraire proprement
UserConsent	Audit/GDPR ou Identity	Découper	Consentement = conformité, rattachement user = identité
AuditLog	Audit/GDPR	Migrer	Doit devenir consommateur d'événements
Enemy	Catalog + Game Engine	Découper	Template catalogue + snapshot runtime
GetScaledEnemy	Game Engine + Catalog + Player	Reconcevoir	Logique utile, endpoint v1 à remplacer
Item	Catalog + Player + Game Engine	Découper	Template, possession durable, reward runtime
Skill	Catalog + Player + Game Engine	Découper	Template, unlock, snapshot combat
Npc / NpcInteraction	Catalog + Game Engine + Player	Découper	Template, event resolver, historique joueur
PlayerProfile	Player	Adapter	Créer snapshot run/combat
PlayerInventory	Player	Migrer	Distinguer permanent vs run temporary
PlayerSkill	Player	Migrer	Déblocages durables
BestiaryUnlock	Player + Catalog	Découper	Unlock joueur + référence ennemi
CombatStats	Player	Renommer	Statistiques cumulées, pas combat runtime
CompanionState	Player / Companion module	Migrer plus tard	Base intéressante pour Neige
GameSave	Event Store / projections	Ne pas migrer tel quel	Remplacé par événements de run

4. Comparaison V1 / V2

Axe	V1 RPG_ESI07	V2 L'épopée des silences	Décision
Client principal	Unity + portail Vue	Web Vue 3 / TypeScript	Unity devient legacy
Gameplay	Progression Unity / sauvegarde classique	Run roguelite procédurale par seed	Repenser GameSave
Backend	API REST Clean Architecture monolithique	Game Engine central + services périphériques	Extraction progressive
Combat	Données récupérées par Unity, validation partielle	Résolution serveur-autoritaire	Nouveau modèle runtime
Contenu	Tables Enemies/Items/Skills/Npcs	Catalog versionné	Transformer en templates
Progression	PlayerProfile porte beaucoup d'état	Player durable + RunSnapshot	Découper
Sauvegarde	GameSave relationnel	Event Store + projections	Remplacer
Narration	PNJ/interactions limitées	Fragments, Tome, Elise, event resolvers	Étendre
Lois	Absentes	Palace Law Engine	Domaine déjà amorcé
Sécurité	JWT, MFA, Argon2id, audit	Identity + Audit/GDPR dédiés	Migrer sans perdre les acquis

5. Procédure de migration progressive



Chaque étape doit produire un commit isolé, des tests et une décision documentée.

Chaque extraction doit être traitée comme un jalon isolé avec documentation, tests, commit dédié et décision explicite.

Étape	Procédure	Critère de sortie
1. Inventaire	Lister entités, handlers, controllers, repositories et tests legacy	Document de mapping validé
2. Bounded context	Classer chaque élément : Catalog, Player, Game Engine, Identity, Audit/GDPR, Leaderboard	Aucun élément critique non classé
3. Catalog minimal	Créer service ou squelette Catalog avec templates essentiels	EnemyTemplate, SkillTemplate, ItemTemplate testés
4. Contrats	Créer les ports entre Game Engine et Catalog	Game Engine ne dépend pas des entités EF Catalog
5. Combat MVP	Créer CombatInstance et CombatantSnapshot	Un combat simple backend-authoritative passe les tests
6. Player extraction	Migrer progression durable	RunSnapshot disponible au démarrage de run
7. Projections	Brancher leaderboard, audit, tome	Événements consommés par projections

6. Procédures détaillées

6.1 Procédure d'extraction d'un domaine

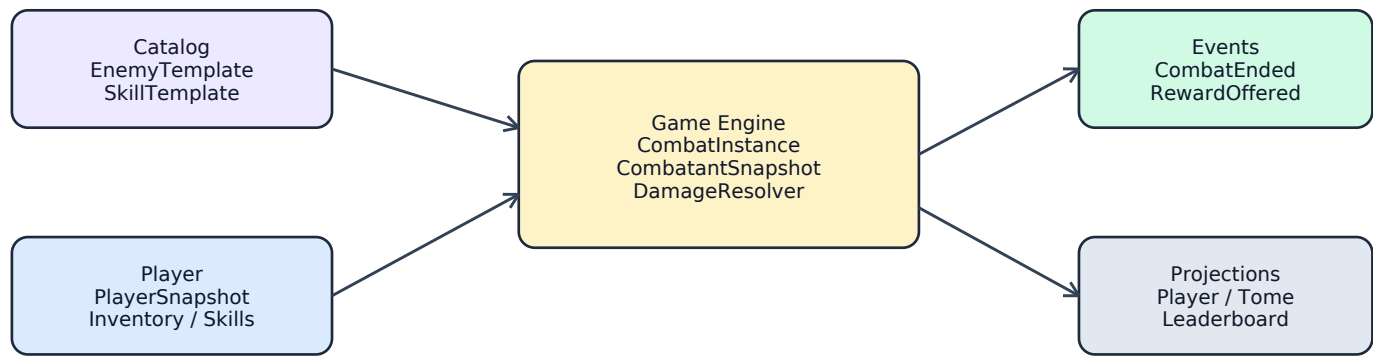
#	Action	Livrable
1	Identifier les classes legacy concernées	Liste dans le document de mapping
2	Identifier les invariants métier utiles	Notes de migration
3	Choisir le service cible	Décision de bounded context
4	Créer le modèle v2 adapté	Entity / ValueObject / DTO / Contract
5	Écrire les tests domaine avant adaptation avancée	Unit tests
6	Créer les ports applicatifs	Interfaces côté Application
7	Implémenter une source temporaire si nécessaire	Static adapter / InMemory adapter
8	Brancher le Game Engine sans couplage direct EF	Contrats ou client interne
9	Documenter les écarts v1/v2	ADR ou suivi technique
10	Commit isolé	Message conventionnel

6.2 Definition of Done migration

Une migration est considérée acceptable lorsque : le modèle v2 a ses propres règles, les tests passent, le service cible est clair, le Game Engine n'a pas de dépendance interdite, les anciennes entités ne sont pas copiées sans transformation, et la décision est documentée.

7. Préparation du Combat MVP

Le combat ne doit pas être codé avant le Catalog minimal, car il dépend des ennemis, compétences, objets, snapshots joueur et lois actives. Le backend v1 fournit des bases utiles, mais pas le modèle runtime final.



Concept	Owner v2	Origine v1	Commentaire
EnemyTemplate	Catalog	Enemy	Stats de base, type, résistances, description
SkillTemplate	Catalog	Skill	Coût, élément, dégâts, soin, effet
PlayerSnapshot	Player / Game Engine	PlayerProfile	Copie contrôlée au démarrage de run/combat
CombatInstance	Game Engine	Nouveau	État complet du combat en cours
CombatantSnapshot	Game Engine	Enemy + PlayerProfile + Skill	Stats effectives runtime
DamageResolver	Game Engine	Nouveau	Résolution autoritaire dégâts / résistances / lois
PlayerCombatStatistics	Player	CombatStats	Statistiques cumulées post-combat

8. Risques et arbitrages

Risque	Impact	Réponse recommandée
Microservices trop tôt	Complexité et ralentissement	Extraire seulement Catalog minimal avant combat
Copie directe v1	Modèle v2 incompatible	Transformer en templates, snapshots et projections
Catalog dans Game Engine	Couplage long terme	Créer un service ou au minimum un bounded context explicite
Combat sans PlayerSnapshot	Mélange runtime/progression durable	Définir snapshot avant DamageResolver
Event Sourcing retardé trop longtemps	Difficulté à tracer les runs	Introduire événements dès que boucle runtime est stabilisée
Identity extrait trop tôt	Perte de temps avant gameplay	Garder legacy temporairement et migrer par priorité sécurité
Audit/GDPR oublié	Dette de conformité	Prévoir consommation d'événements dès les contrats
Compagnon Markov perdu	Perte d'un concept utile	Documenter et réintroduire dans Player/Companion plus tard

9. Roadmap recommandée

Ordre	Jalon	Type	Commit recommandé
1	Mapping backend legacy vers services v2	Documentation	docs(v2): map legacy backend domains to target services
2	Catalog Service skeleton	Architecture	feat(catalog): initialize catalog service skeleton
3	Enemy/Skill/Item templates	Catalog	feat(catalog): add core gameplay templates
4	Contrats Game Engine ↔ Catalog	Intégration	feat(game-engine): add catalog content contracts
5	Combat domain minimal	Game Engine	feat(game-engine): introduce combat runtime domain
6	Combat action endpoint	API	feat(game-engine): expose combat action endpoint
7	Reward offer/select	Game Engine	feat(game-engine): expose reward selection flow
8	Player snapshot	Player/Game Engine	feat(player): expose run player snapshot
9	Event Store run events	Infrastructure	feat(game-engine): persist run events
10	Projection leaderboard / tome	Projections	feat(leaderboard): project completed runs

Conclusion

Le passage V1 vers V2 doit être traité comme une migration de connaissances, pas comme une réécriture impulsive. Le backend legacy a une vraie valeur : il documente les premiers choix, les entités métiers, les exigences sécurité et plusieurs mécanismes utiles. La V2 doit les absorber proprement dans une architecture plus claire, plus serveur-autoritaire et plus adaptée au roguelite narratif.